

Aether — Deployment Environments

Dev, Staging, Production — Managed as Code

Create isolated environments, promote workloads between tiers with automatic resource scaling, and validate environment parity to prevent surprises in production.

Development • Staging • Production • Custom

Environment Management

Aether's EnvironmentManager provides structured, tier-aware environments with per-environment workloads, variables, and resource overrides.



Development

Minimal resources, fast iteration. Single replicas, relaxed policies. Aliases: dev, development. Ideal for local Podman runtimes with rapid rebuild cycles.



Staging

Production-like configuration with moderate resources. Aliases: stg, staging. Used for integration testing, performance validation, and pre-release verification.



Production

Full resources, strict policies, multi-replica. Aliases: prod, production. Policy engine enforces min replicas, health checks, and resource limits.

Creating and Listing Environments

```
# Create environments
$ aether env create dev --tier development
$ aether env create staging --tier staging
$ aether env create prod --tier production
$ aether env create qa-team --tier custom

# List all environments
$ aether env list
```

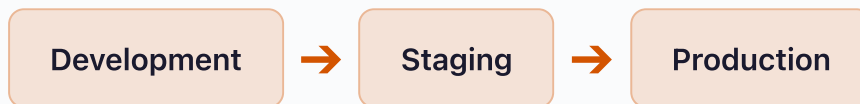
Environment	Tier	Workloads	Variables	Updated
dev	development	3	5	2026-04-10T08:00
prod	production	3	8	2026-04-12T14:00
qa-team	qa-team	1	2	2026-04-11T09:00
staging	staging	3	6	2026-04-11T16:00

Per-Environment Configuration

Each environment stores workloads with optional overrides for runtime, replicas, CPU, memory, and environment variables. This allows the same workload spec to run with 1 replica on Podman in dev but 4 replicas on Kubernetes in production, without maintaining separate YAML files.

Workload Promotion

Promote workloads between environments with automatic tier-aware adjustments. Aether handles version bumps, resource scaling, and production readiness checks.



Promotion Strategies



Direct

Copy the workload spec as-is to the target environment. No resource adjustments. Version is bumped automatically. Best for environments at the same tier.



Tier-Adjusted

Automatically scale resources based on tier transition. Dev-to-Prod applies a 4x multiplier to replicas. Staging-to-Prod applies 2x. Production warnings for insufficient replicas.



Canary

Deploy to the target environment using canary deployment strategy. Traffic is gradually shifted to the new version with automatic rollback on failure.

```
# Promote from dev to staging (direct copy)
$ aether env promote api-service --from dev --to staging --strategy direct

Promotion: api-service (dev -> staging)
[change] version: 1.0.0 -> 1.0.1 (Version bump for promotion)
Success: api-service is now in staging

# Promote from dev to production (tier-adjusted)
$ aether env promote api-service --from dev --to prod --strategy tier-adjusted
```

```
Promotion: api-service (dev -> prod)
[change] replicas: 1 -> 4 (Tier adjustment: development -> production)
[change] version: 1.0.0 -> 1.0.1 (Version bump for promotion)
Success: api-service is now in prod
```

Tier Scaling Factors

Transition	Scale Factor	Example: 1 replica
Dev → Staging	2x	1 → 2 replicas
Dev → Production	4x	1 → 4 replicas
Staging → Production	2x	2 → 4 replicas
Production → Staging	0.5x	4 → 2 replicas
Production → Dev	0.25x	4 → 1 replica

Environment Parity

Compare two environments to find differences that could cause production surprises. Aether classifies each difference by severity to help teams prioritize fixes.

```
$ aether env parity --env-a staging --env-b prod --workload api-service
```

```
Parity: staging vs prod for 'api-service'
```

```
Status: DIFFERENCES FOUND
```

```
[expected]  version:  1.0.0 vs 1.0.1
[expected]  replicas: 2 vs 4
[expected]  cpu:      2 vs 4
[unexpected] env.LOG_LEVEL: debug vs (not set)
[CRITICAL]  env.DB_HOST: staging-db.internal vs (not set)
```

Parity Severity Levels



Expected

Differences that are normal between tiers: replica count, CPU/memory limits, version numbers. These differences are intentional and represent proper tier scaling.



Unexpected

Differences that deserve investigation: environment variables that exist in one environment but not the other, runtime overrides that diverge without clear reason.



Critical

Workload exists in one environment but is completely missing from the other. This indicates a failed promotion or accidental deletion that requires immediate attention.

What Parity Checks Compare

Field	Check Type	Typical Severity
Version	String equality	Expected (if different tiers)
Runtime Override	Option equality	Unexpected
Replicas	Numeric equality	Expected
CPU / Memory	Option equality	Expected
Environment Variables	Key-by-key comparison	Unexpected
Workload Existence	Present in both	Critical

Environment Isolation

Each environment operates as an independent configuration domain with its own workloads, variables, and deployment state. Changes in one environment never affect another.



Configuration Isolation

Each environment stores its own workload configurations with independent runtime overrides, replica counts, and resource limits. Promoting a workload creates a copy — it never shares a reference.



Variable Isolation

Environment variables are scoped to each environment. `DB_HOST` in `dev` points to `localhost`, while production points to `prod-db.internal`. No variable leakage between tiers.



Version Tracking

Each workload in each environment tracks its own version independently. Promotions automatically bump the version, creating a clear audit trail of what version is deployed where.



Policy per Tier

The Aether policy engine enforces tier-specific rules. Production requires min 2 replicas and health probes. Development allows single replicas and optional health checks.

```
# Set environment-specific variables
$ aether env set-var dev DB_HOST localhost
$ aether env set-var staging DB_HOST staging-db.internal
$ aether env set-var prod DB_HOST prod-db.internal

# Add workload with environment-specific overrides
$ aether env add-workload prod api-service \
```



```
--spec api.yaml \  
--replicas 4 \  
--cpu 4 \  
--memory 8Gi
```



Environment isolation is not optional — it is the foundation that makes promotion workflows safe. When dev and prod share configuration, every deploy is a risk.

Ship with Confidence

From first commit to production deploy, Aether environments give every workload a clear path forward.

3+

Environment Tiers

3

Promotion Strategies

3

Parity Severity Levels

Structured Deployment Pipelines

Create environments, promote workloads with tier-aware scaling, validate parity before production, and maintain full isolation — all from the Aether CLI.

```
# Complete environment workflow
$ aether env create dev --tier development
$ aether env create prod --tier production
$ aether env add-workload dev api --spec api.yaml --replicas 1
$ aether env promote api --from dev --to prod --strategy tier-adjusted
$ aether env parity --env-a dev --env-b prod --workload api
```